

End-to-End Data Analytics Project: Blinkit

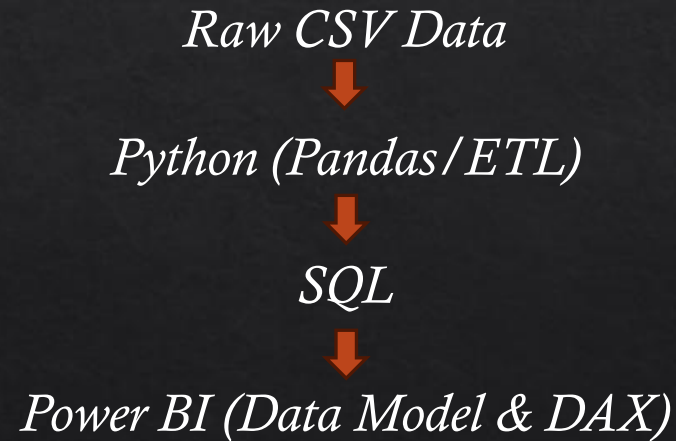
Retail & Supply Chain Optimization

Data Pipeline, Inventory and Sales Insights

Presented By : Kushang Bhattacharya

Project Architecture & Core Metrics

- Developed a full-stack data pipeline from raw transaction files to interactive business intelligence dashboards.
- Driven by a dataset encompassing 5,000+ Total Orders, 2,172 Unique Customers, and 268 Active SKUs, yielding a Gross Revenue of ₹4.97 Million.
- Core focus areas: Sales velocity tracking, inventory depletion risks, and logistics fulfillment metrics.



Data Cleaning, Transformation & Database Ingestion

- Extracted three source datasets (products, orders, order_items) and profiled schemas via pandas (.shape, .info()).
- Cleansed data by handling structural nulls and eliminating duplicate rows to maintain baseline integrity.
- Converted mission-critical temporal columns (order_date, promised_delivery_time, actual_delivery_time) from object strings to structured datetime format.

```
orders_df['order_date'] = pd.to_datetime(orders_df['order_date'])
orders_df['promised_delivery_time'] = pd.to_datetime(orders_df['promised_delivery_time'])
orders_df['actual_delivery_time'] = pd.to_datetime(orders_df['actual_delivery_time'])

orders_df.info()
```

- Automated database schema seeding into MySQL via sqlalchemy and pymysql with index=False.

```
products_df.to_sql('products', con=engine, if_exists='replace', index=False)

orders_df.to_sql('orders', con=engine, if_exists='replace', index=False)

orders_items_df.to_sql('order_items', con=engine, if_exists='replace', index=False)
```

Database Engineering & Analytical Profiling

- Created a dedicated production repository named `blinkit_db` to handle cross-table relational dependencies.
- Structured and executed 7 high-impact diagnostic queries to validate post-ETL data distribution and business health.
- Formulated foundational aggregations including product density checks, top margin SKUs, monthly trends, and preferred checkout networks.


```

1 • create database blinkit_db;
2
3 • SELECT
4     COUNT(*) AS total_products
5 FROM
6     products;
7
8 -- Catagory wise product count - dairy & breakfast are at top
9
10 • SELECT
11     category, COUNT(*) AS total_items
12 FROM
13     products
14 GROUP BY category
15 ORDER BY total_items DESC;
16
17 -- Top 5 most expensive products - Vitamins in pharmacy category is the most expensive products
18 • SELECT
19     product_name, price, category
20 FROM
21     products
22 ORDER BY price DESC
23 LIMIT 5;
24
25
26 -- Total Revenue - 4972415.43 /-

```

```

45 -- Monthly Sales Trend - (August , May , oct , sep)
46
47
48 • SELECT
49     MONTHNAME(orders.order_date) AS month_name,
50     ROUND(SUM(order_items.quantity * order_items.unit_price),
51           2) AS monthly_revenue
52 FROM
53     order_items
54     JOIN
55     orders ON order_items.order_id = orders.order_id
56 GROUP BY MONTHNAME(orders.order_date) , MONTH(orders.order_date)
57 ORDER BY MONTH(orders.order_date);
58

```

```

28 • SELECT
29     ROUND(SUM(quantity * unit_price), 2)
30 FROM
31     order_items;
32
33 -- Revenue by Category - dairy & breakfast is at top and pharmacy is at second position.
34 • SELECT
35     products.category,
36     ROUND(SUM(order_items.quantity * order_items.unit_price),
37           2) AS category_revenue
38 FROM
39     order_items
40     JOIN
41     products ON order_items.product_id = products.product_id
42 GROUP BY products.category
43 ORDER BY category_revenue DESC;
44

```

```

60 -- Most Preferred Payment Methods
61
62 • SELECT
63     payment_method, COUNT(order_id) AS total_orders
64 FROM
65     orders
66 GROUP BY payment_method
67 ORDER BY total_orders DESC;
68

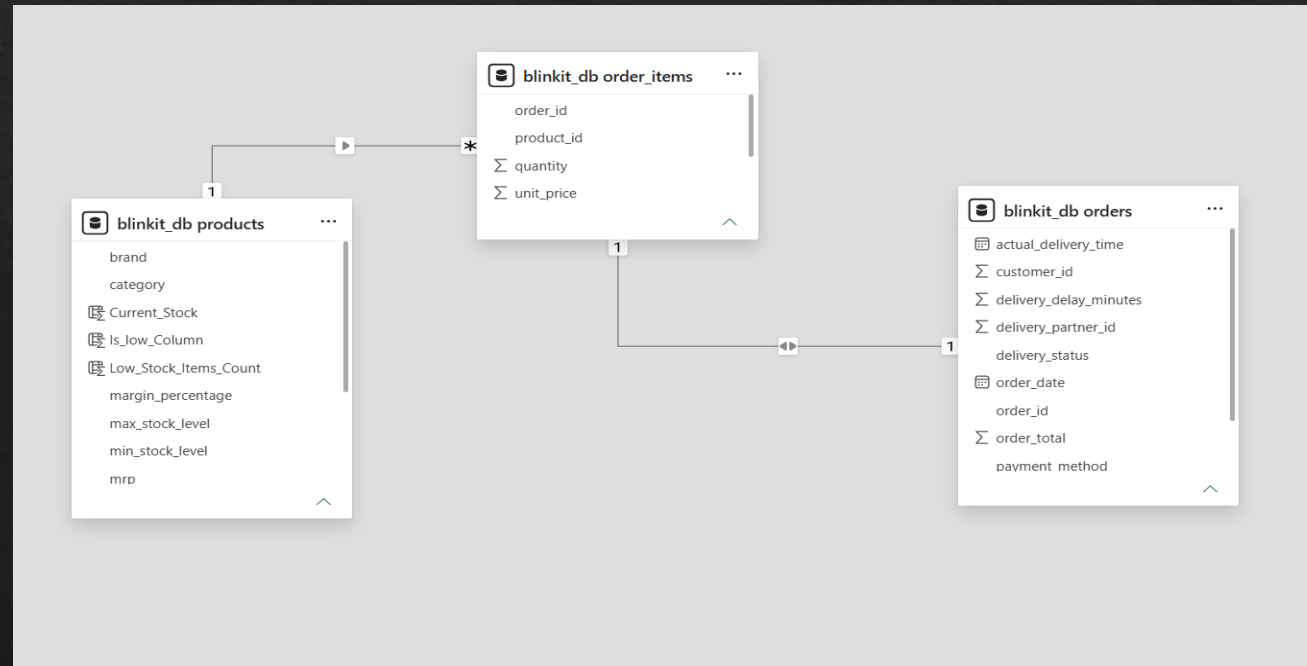
```

Relational Data Modeling & Schema Design

Engineered a highly optimized Star Schema data model to eliminate redundancy and support fast query rendering.

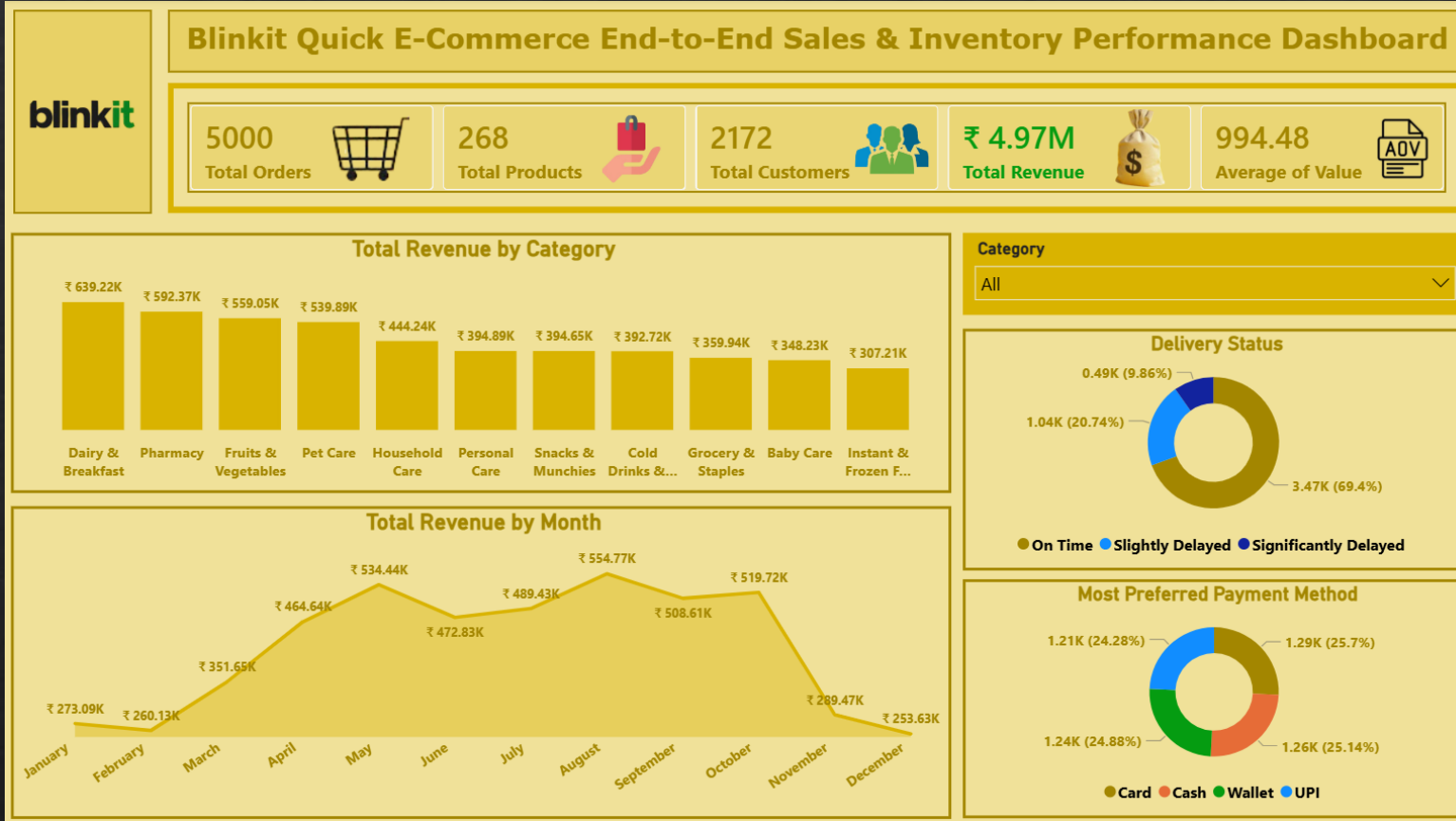
Established strict One-to-Many relationships between lookup dimensions (products, orders) and the transactional fact table (order_items).

Enabled cross-filtering directions to ensure consistent slice-and-dice behavior across diverse operational visuals.



Dashboard Page 1: Executive Overview & KPIs

- **Core KPIs Captured:** Total Sales at ₹4.97 Million, Average Order Value (AOV) optimized at ₹994.98.
- **Category Drivers:** Essential baskets like Dairy & Breakfast (₹639.22K) and Pharmacy (₹592.37K) emerged as top revenue pillars.
- **Seasonality Trends:** Sales peaked sharply between May and October (festival/monsoon demand), while hitting a critical bottleneck in December, January and February.
- **Checkout & Logistics:** Achieved a solid 69.4% On-Time Delivery rate. Payment distribution remained closely competitive, with Card Payments slightly outperforming UPI and Cash.



Monitored high-velocity fulfillment tracking 10,034 Total Units Sold while flagging 41 critical low-stock items.

Formulated a centralized matrix grouping Product, Category, Current Stock, and Safety Thresholds for automated reorder visibility.

Demand Mismatch Alert: Uncovered a severe stock-out risk in Dairy & Breakfast; despite massive customer pull (>1,000 units sold), inventory levels dipped dangerously low (20-30 units).

Leveraged advanced filtering via dynamic category , months and shelf-life dropdowns and modified visual interactions.



Analytical Engineering via DAX Measures

- Developed custom DAX measures to calculate core business KPIs and handle relational table filters.
 - Used standard functions like SUMX, CALCULATE, and DIVIDE for accurate metrics.
- Average of Value = `DIVIDE([Total_Revenue], [Total_Orders], 0)`
 - Total_Orders = `COUNT('blinkit_db order_items'[order_id] )`
 - Total_Revenue = `sumx('blinkit_db order_items','blinkit_db order_items'[quantity]*'blinkit_db order_items'[unit_price])`
 - Total Quantity Sold = `sum('blinkit_db order_items'[quantity])`
 - Current_Stock = `'blinkit_db products'[max_stock_level] - CALCULATE(SUM('blinkit_db order_items'[quantity]))`
 - Is_low_Column = `IF('blinkit_db products'[Current_Stock]<'blinkit_db products'[min_stock_level],1,0)`
 - Low_Stock_Items_Count = `CALCULATE(COUNT('blinkit_db products'[product_id]), 'blinkit_db products'[Is_low_Column] = 1)`

Project Conclusion & Key Takeaways

- **Inventory Management:** Dairy & Breakfast category needs instant stock alert triggers as its sales are extremely high but current stock depths are critically low.
- **Fulfillment Efficiency:** Delivery operations are running smoothly with a strong 69.4% On-Time Delivery rate.
- **Payment Systems:** Payment modes are evenly balanced, with Credit/Debit cards being slightly more preferred by customers.

Project Links & Contact

Connect with me:

- **LinkedIn Profile Link** : [Click here](#) (*Watch the project walkthrough video on my profile*)
- **GitHub Repository Link** : [Click here](#) (*Access the full Python code, SQL scripts, and Dataset here*)
- **Video Presentation Link** : [Watch Video](#)

Thank You